

IF2224 TEORI BAHASA FORMAL DAN OTOMATA
LAPORAN TUGAS BESAR
MILESTONE 1

<FOTO KELOMPOK>

Disusun oleh:

Kelompok 01 – Kelas 01

Muhammad Jordan Ferimeison (13524047)

Arina Azka (13524049)

Hakam Avicena Mustain (13524075)

Yavie Azka Putra Araly (13524077)

Angelina Andra Alanna (13524079)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
LANDASAN TEORI.....	3
1.1. Lexical Analyzer.....	3
1.2. Token.....	3
1.3. Lexer.....	3
1.4. Deterministic Finite Automata (DFA).....	3
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	4
2.1. Perancangan Sistem (?).....	4
2.2. Diagram DFA.....	4
2.3. Implementasi Program.....	4
2.4. Implementasi Lexical Analyzer.....	4
2.5. Implementasi DFA.....	4
BAB III	
PENGUJIAN.....	5
4.1. Metode Pengujian.....	5
4.2. Hasil Pengujian.....	5
4.1. Kasus Uji 1.....	5
4.2. Kasus Uji 2.....	5
4.3. Kasus Uji 3.....	5
4.4. Kasus Uji 4.....	5
4.5. Kasus Uji 5.....	5
BAB IV	
KESIMPULAN DAN SARAN.....	6
4.1. Kesimpulan.....	6
4.2. Hasil Pengujian.....	6
DAFTAR PUSTAKA.....	7

BAB I

LANDASAN TEORI

1.1. Lexical Analyzer

Lexical Analyzer, atau yang dikenal sebagai scanner, merupakan tahap pertama dalam proses kompilasi suatu bahasa pemrograman. Komponen ini bertugas membaca karakter-karakter source code dari kiri ke kanan, kemudian mengelompokkannya menjadi satuan-satuan makna yang disebut token. Pada dasarnya, lexical analyzer berfungsi sebagai antarmuka antara source code mentah dan analisis sintaksis (syntax analysis).

Secara umum, lexical analyzer menjalankan tiga fungsi utama, yaitu:

1. Membaca karakter satu per satu dari input source code
2. Mengelompokkan karakter-karakter tersebut menjadi token yang bermakna.
3. Membuang elemen yang tidak relevan seperti spasi, baris baru, dan komentar.

Dalam konteks pembuatan compiler untuk bahasa pemrograman Arion, lexical analyzer diimplementasikan menggunakan Deterministic Finite Automata (DFA), yaitu sebuah model matematis yang mendefinisikan aturan transisi antar state berdasarkan karakter yang dibaca.

1.2. Token

Token merupakan unit leksikal terkecil yang memiliki makna dalam sebuah bahasa pemrograman. Setiap token terdiri dari dua komponen yaitu tipe (*type*) yang merepresentasikan kategori token, dan nilai (*value*) yang merupakan string aktual dari token tersebut dalam source code. Token dihasilkan oleh lexical analyzer dan kemudian digunakan sebagai masukan untuk tahap analisis sintaksis.

Dalam bahasa pemrograman Arion, terdapat 52 jenis token yang dikategorikan ke dalam beberapa kelompok, yaitu: konstanta (*intcon*, *realcon*, *charcon*, *string*), operator aritmatika dan logika, operator perbandingan, tanda baca, kata kunci (*keyword*), identifier (*ident*), dan komentar (*comment*).

<i>Tabel 1.1 - Contoh Token Dalam bahasa Pemrograman Arion</i>			
Token	Tipe	Nilai	Contoh dalam Kode
intcon	Konstanta Integer	5	a := 5
realcon	Konstanta Real	3.14	pi := 3.14
ident	Identifier	myVar	var Myvar: integer
becomes	Assignment	:=	x := 10
plus	Operator +	+	a + b
beginsy	Keyword	begin	begin ... end
iparent	Tanda Kurung	(	writeln(x)
semicolon	Tanda Baca	;	a := 5

Beberapa token bersifat case-insensitive, Seperti: begin, BEGIN, Begin akan dikenali sebagai token yang sama, yaitu beginsy. Sementara itu, token ident (identifier) harus diawali dengan huruf dan dapat diikuti kombinasi huruf dan angka.

1.3. Lexer

Lexer (juga disebut tokenizer atau scanner) adalah komponen perangkat lunak yang mengimplementasikan proses lexical analysis. Lexer membaca karakter masukkan dari kiri ke kanan secara sekuensial dan menerapkan aturan pengenalan pola untuk menghasilkan token-token yang sesuai. Ketika suatu pola karakter cocok dengan aturan tertentu, lexer membuat sebuah token dan menyimpannya untuk diproses lebih lanjut.

Dalam implementasinya, lexer bekerja melalui beberapa tahapan. Pertama, lexer membaca satu karakter dari input. Kemudian, berdasarkan karakter tersebut, lexer menentukan state berikutnya sesuai dengan aturan DFA yang telah didefinisikan. Proses ini berlanjut hingga lexer mencapai accepting state, yaitu kondisi di mana satu token berhasil dikenali secara lengkap. Karakter pertama yang tidak termasuk dalam token tersebut akan dikembalikan (*backtrack*) untuk diproses sebagai awal token berikutnya.

Selain mengenali token yang valid, lexer juga bertugas menangani kondisi *error*, yaitu ketika ditemukan karakter atau pola yang tidak sesuai dengan bahasa yang berlaku. Dalam kondisi ini, lexer diharapkan mampu menghasilkan pesan kesalahan yang informatif tanpa menyebabkan program berhenti secara paksa.

<i>Tabel 1.2 - Cara Kerja Lexer dalam Pseudo-code</i>
<code>state = START</code>
<code>buffer = ""</code>
<code>while (belum EOF):</code>
<code>ch = baca_karakter_berikutnya()</code>
<code>state = transisi_DFA(state, ch)</code>
<code>buffer += ch</code>
<code>if (state == ACCEPT):</code>
<code>emit_token(state, buffer)</code>
<code>buffer = ""</code>
<code>state = START</code>
<code>elif (state == ERROR)</code>
<code>cetak_error(ch)</code>
Dalam C++, hal ini biasanya diimplementasikan sebagai fungsi <code>nextToken()</code> yang dipanggil berulang oleh parser. Setiap pemanggilan menghasilkan satu token berikutnya dari input.

1.4. Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) adalah model komputasi matematis yang digunakan untuk mengenali pola dalam suatu string masukan. DFA didefinisikan secara formal sebagai tupel lima Komponen: $M = (Q, \Sigma, \delta, q_0, F)$, di mana:

- **Q** : himpunan state yang terdefinisi (finite)
- **Σ** : alfabet, yaitu himpunan simbol yang valid sebagai masukan
- **δ** : fungsi transisi, yaitu $\delta: Q \times \Sigma \rightarrow Q$ yang menentukan state berikutnya

- q_0 : state awal (initial state), di mana $q_0 \in Q$
- F : himpunan state penerima (accepting states), di mana $F \subseteq Q$

Sifat deterministic pada DFA berarti bahwa untuk setiap state dan setiap karakter masukan, terdapat satu transisi yang mungkin terjadi. Ini berbeda dengan NFA (*Non-deterministic Finite Automata*) yang memungkinkan lebih dari satu transisi atau transisi tanpa membaca input (ϵ -transition). Meskipun keduanya setara secara ekspresif, DFA lebih unggul dalam implementasi karena setiap input hanya memiliki satu jalur transisi, sehingga tidak memerlukan backtracking maupun pencarian jalur alternatif.

Berikut merupakan contoh tabel transisi DFA untuk beberapa token dalam bahasa pemrograman Arion. Tabel ini menggambarkan bagaimana lexer berpindah dari satu state ke state lainnya saat membaca karakter input satu per satu

Keterangan Kolom:

- **State**, posisi lexer saat ini sebelum membaca input
- **Input**, karakter yang sedang dibaca
- **Next State**, posisi lexer setelah input dibaca
- **Accept**, apakah buffer sebelum input ini masuk sudah membentuk token yang valid. Jika Ya, lexer siap mengeluarkan token jika karakter berikutnya tidak cocok. Jika Tidak, berhenti di sini berarti error.
- **Token**, jenis token yang siap dikeluarkan jika Accept = Ya
- **Buffer**, isi karakter yang sudah terkumpul setelah input diproses
- **Keterangan**, penjelasan kondisi buffer sebelum input masuk

<i>Tabel 1.3 - Contoh Tabel Transisi DFA untuk Token intcon, realcon, dan ident</i>						
State	Input	Next State	Accept	Token	Buffer	Keterangan
Konstanta Integer $\rightarrow$ intcon						
S0	Digit	S_INT	Tidak	-	"4"	S0 mulai kumpul angka

S_INT	Digit	S_INT	Ya	intcon	"48"	Buffer nambah, intcon valid
Konstanta Rill → realcon						
S_INT	.	S_REAL_DOT	Ya	intcon	"48."	S_INT valid tapi lanjut cek desimal
S_REAL_DOT	Digit	S_REAL	Tidak	-	"48.1"	Baru punya 48., belum valid
S_REAL	Digit	S_REAL	Ya	realcon	"48.14"	Buffer valid sebagai realcon
Operator Tunggal → plus						
S0	+	S_PLUS	Tidak	-	"+"	Buffer kosong sebelum input → belum ada token
S_PLUS	(sel esai)	-	Ya	plus	"+"	Buffer "+" sebelum selesai → plus valid
Tanda baca & Assignment → colon/becomes						
S0	:	S_COLON	Tidak	-	":"	Buffer kosong sebelum input → belum ada token
S_COLON	=	S_BECOMES	Ya	colon	":="	Buffer ":" sebelum input = → colon valid
S_BECOMES	(sel esai)	-	Ya	becomes	":="	Buffer "":=" sebelum selesai → becomes valid
Identifier Keyword → Ident/keyw						
S0	huruf	S_IDENT	Tidak	-	"b"	Buffer kosong sebelum input → belum ada token
S_IDENT	huruf/digit	S_IDENT	Ya	ident*	"be", "beg"...	Buffer "b" sebelum input → ident valid, terus kumpul
S_IDENT	(sel esai)	cek keyword	Ya	keyword/ ident	"begin"	Buffer "begin" sebelum selesai → cek tabel keyword

1.5. Hubungan Antar Konsep

Keempat konsep yang telah dijelaskan di atas saling berkaitan erat dalam proses kerja sebuah compiler.

- Lexical Analyzer adalah tahapan proses secara keseluruhan
- Lexer adalah komponen perangkat lunak yang mengimplementasikan tahapan tersebut
- DFA adalah model matematis yang menjadi landasan kerja lexer
- Token adalah keluaran yang dihasilkan oleh lexer sebagai hasil dari proses tersebut.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Perancangan Sistem

Perancangan sistem lexical analyzer dimulai dari membaca dan memahami pola karakter untuk menghasilkan token dari masukan bahasa pemrograman Arion. Pola karakter tersebut kemudian dimodelkan sebagai Deterministic Finite Automata (DFA) yang dapat menerima atau menolak suatu string input berdasarkan state-nya.

DFA tersebut direpresentasikan secara grafis sebagai grafik terarah dengan *node* mewakili suatu *state* dan *edge* mewakili transisi antar status. Setiap *edge* diberi simbol input dengan panah yang menunjukkan perubahan dari satu state ke state lainnya. Diagram tersebut menjadi acuan utama untuk implementasi *source code* dalam bahasa C++ GNU. Sayangnya, kode tidak dapat diimplementasikan persis seperti diagram DFA sehingga dilakukan beberapa penyesuaian. Implementasi kode persis seperti diagram DFA dapat membuat kode menjadi terlalu kompleks, sulit dipahami, dan sulit dimodifikasi ketika diperlukan.

Perbedaan diagram DFA dan implementasi kode terletak pada cara pembacaan keyword bahasa Arion dan state yang digunakan. Pada DFA, input benar-benar dibaca satu persatu karakter, setiap pembacaan satu karakter akan mengakibatkan perubahan transisi. Namun, pada implementasi kode, state tidak berubah tiap pembacaan satu huruf, tetapi tiap pembacaan satu string.

Meskipun demikian, perbedaan antara diagram DFA dan implementasi kode tidak mempengaruhi alur kerja utama program. Alur utama program dapat dilihat pada kode fungsi `main()`. Pertama, fungsi tersebut akan membaca menerima argumen pengguna lalu menangani pembacaan file input. Hasil pembacaan file input tersebut selanjutnya diserahkan ke objek Lexer untuk diproses menjadi daftar token. Hasil tokenisasi tersebut kemudian ditulis ke file output. Detail implementasi lebih lanjut dibahas pada bagian-bagian berikutnya.

2.2. Implementasi DFA

2.2.1. Representasi DFA

Pada milestone ini, DFA direpresentasikan sebagai suatu diagram yang dapat diakses melalui link di bawah ini.

https://miro.com/app/board/uXjVGusf64o=?share_link_id=372355156236

Node dalam grafik tersebut memiliki warna yang berbeda untuk memudahkan pembaca dalam mengenali jenis dari suatu state seperti berikut.

Warna State	Penjelasan
Hijau Double Circle	Accepting State
Abu-abu single Circle	Intermediate state (transisi, belum, menghasilkan token)
Merah	S_ERROR (input tidak valid)

Simbol

- $\rightarrow$ START = s0 adalah state awal
- Garis putus-putus = token yang dikembalikan
- Self-loop = membaca karakter berulang (misal digit terus-menerus)

Alur Per Kategori

a. Numerik

DFA membaca digit dari s0 lalu masuk ke s1 yang merupakan accepting state untuk intcon. Selama karakter berikutnya masih digit, s1 melakukan self-loop. Jika ditemukan titik setelah s1, DFA berpindah ke s2 sebagai intermediate state kandidat realcon. Dari s2, jika diikuti digit maka masuk ke s3 sebagai accepting state untuk realcon. Namun jika s2 tidak diikuti digit, DFA langsung menuju S_ERROR.

b. Tanda Baca & Operator Tunggal

Karakter tunggal seperti ,, ;, ., +, -, *, / dari s0 langsung menuju masing-masing accepting state dan mengembalikan tokennya. Untuk karakter :, DFA masuk ke intermediate state terlebih dahulu — jika diikuti = maka menghasilkan becomes, jika tidak maka menghasilkan

colon. Operator < dan > juga bersifat serupa, di mana < bisa berlanjut menjadi leq atau neq, sedangkan > bisa berlanjut menjadi geq. Tanda = tunggal selalu menuju S_ERROR karena Arion mengharuskan == untuk operator equal.

c. String, Charcon, dan Komentar

Ketika s0 membaca karakter ', DFA mulai mengakumulasi isi literal. Saat penutup ' ditemukan, DFA menentukan jenis token berdasarkan panjang buffer — kosong berarti string, panjang 1 berarti charcon, dan lebih dari 1 berarti string. Jika EOF atau newline ditemukan sebelum penutup, DFA menuju S_ERROR. Untuk komentar, pola {...} dibaca hingga } ditemukan, sedangkan pola (*...*) dibaca hingga *) ditemukan. Keduanya menghasilkan token comment, dan keduanya juga menuju S_ERROR jika tidak ditutup sebelum EOF.

d. Identifier dan Keyword

Penjelasan bagian identifier dan keyword yang tadi perlu diperbarui menjadi yang DFA membaca huruf pertama dari s0, lalu mengikuti jalur transisi per karakter sesuai pola keyword yang mungkin. Jika di titik manapun karakter tidak cocok dengan kelanjutan keyword, DFA langsung menuju S_IDENT dan mengembalikan token ident. Jika seluruh karakter cocok hingga state akhir jalur keyword tersebut, maka token keyword yang sesuai dikembalikan.

2.2.2. DFA pada Kode Program

Representasi DFA dalam diagram yang membaca input satu demi satu karakter mengakibatkan jumlah state yang sangat banyak. Maka dari itu, state yang digunakan pada diagram berbeda dengan state yang digunakan pada kode program. Berikut ini adalah implementasi state DFA yang digunakan pada kode program.

a. Daftar State DFA

State	Fungsi
S0	State awal

S_INT	Membaca bilangan bulat
S_REAL_DOT	Sudah membaca titik setelah integer
S_REAL	Membaca bagian pecahan bilangan riil
S_QUOTE	String selesai atau string kosong
S_CHAR	Konstanta karakter tunggal
S_IDENT	Identifier
S_PLUS	Operator +
S_MINUS	Operator -
S_TIMES	Operator *
S_RDIV	Operator /
S_EQL1	Sudah membaca = pertama
S_EQL2	Sudah membaca ==
S_LESS	Sudah membaca <
S_LESSEQUAL	Sudah membaca <=
S_NOTEQUAL	Sudah membaca <>
S_GREATER	Sudah membaca >
S_GREATEREQUAL	Sudah membaca >=
S_COLON	Sudah membaca :
S_BECOMES	Sudah membaca :=
S_LPAR	Tanda kurung buka (
S_RPAR	Tanda kurung tutup)
S_LBRACK	Tanda kurung siku buka [
S_RBRACK	Tanda kurung siku tutup]
S_COMMA	Tanda koma ,
S_SEMICOLON	Tanda titik koma ;
S_PERIOD	Tanda titik .
S_CMT1	Komentar { ... }
S_CMT2	Komentar (* ... *)
S_CMT2_STAR	Di dalam (* ... *), baru membaca *

S_KEY_INPUT	Sedang membaca kandidat keyword/identifier
S_KEY_CLASSIFY	Siap klasifikasi keyword atau identifier
S_KEYWORD	Keyword
S_ERROR	Error leksikal

b. Transisi DFA untuk Konstanta Numerik

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	digit	S_INT	Lanjut	-	Mulai membaca integer
S_INT	digit	S_INT	Lanjut	-	Digit terus dikumpulkan
S_INT	.	S_REAL_DOT	Lanjut	-	Kandidat bilangan riil
S_REAL_DOT	digit	S_REAL	Lanjut	-	Bagian pecahan valid
S_REAL	digit	S_REAL	Lanjut	-	Pecahan terus dibaca
S_INT	selain digit dan .	final	Final	intcon	Integer selesai
S_REAL	selain digit	final	Final	realcon	Bilangan riil selesai
S_REAL_DOT	selain digit	S_ERROR	Error	error	Titik tidak diikuti digit

c. Transisi DFA untuk String dan Char

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	'	proses string dimulai	Lanjut	-	Awal string/char
awal pembacaan string	karakter biasa selain ', newline, EOF	tetap membaca isi	Lanjut	-	Isi literal ditambahkan ke buffer

awal pembacaan string	newline atau EOF sebelum penutup	S_ERROR	Error	error	Unterminated string
sedang membaca isi	' lalu karakter berikutnya juga '	tetap membaca isi	Lanjut	-	Escape kutip tunggal; menambahkan ' ke buffer
sedang membaca isi	' penutup dan buffer kosong	S_QUOTE	Final	string	String kosong
sedang membaca isi	' penutup dan panjang buffer = 1	S_CHAR	Final	charcon	Konstanta karakter
sedang membaca isi	' penutup dan panjang buffer > 1	S_QUOTE	Final	string	String biasa

d. Transisi DFA untuk Komentar {...}

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	{	mulai proses komentar	Lanjut	-	Awal komentar kurung kurawal
proses komentar {...}	selain } dan EOF	tetap membaca isi	Lanjut	-	Isi komentar masuk buffer
proses komentar {...}	}	S_CMT1	Final	comment	Komentar selesai
proses komentar {...}	EOF	S_ERROR	Error	error	Unterminated comment

e. Transisi DFA untuk Komentar (*...*)

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	(*	mulai proses komentar	Lanjut	-	Awal komentar gaya (* ... *)

proses komentar (*...*)	karakter biasa selain * dan EOF	tetap membaca isi	Lanjut	-	Isi komentar dibaca
proses komentar (*...*)	*	S_CMT2_STA R	Lanjut	-	Kandidat penutup komentar
S_CMT2_STA R	)	S_CMT2	Final	comment	Komentar selesai
S_CMT2_STA R	selain) dan bukan EOF	kembali membaca isi	Lanjut	-	* dianggap bagian isi komentar
proses komentar (*...*)	EOF	S_ERROR	Error	error	Unterminated comment

f. Transisi DFA untuk Operator

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	+	S_PLUS	Final	plus	Operator tambah
S0	-	S_MINUS	Final	minus	Operator kurang
S0	*	S_TIMES	Final	times	Operator kali
S0	/	S_RDIV	Final	rdiv	Operator bagi
S0	=	S_EQL1	Lanjut	-	Kandidat ==
S_EQL1	=	S_EQL2	Final	eql	Operator ==
S_EQL1	selain =	S_ERROR	Error	error	= tunggal dianggap error
S0	<	S_LESS	Lanjut/Final	lss	Dapat menjadi <, <=, atau <>
S_LESS	=	S_LESSEQUAL	Final	leq	Operator <=
S_LESS	>	S_NOTEQUAL	Final	neq	Operator <>
S_LESS	selain = dan >	final	Final	lss	Operator <
S0	>	S_GREATER	Lanjut/Final	gtr	Dapat menjadi > atau >=

S_GREATER	=	S_GREATEREQ	Final	geq	Operator >=
S_GREATER	selain =	final	Final	gtr	Operator >
default	karakter tak sesuai	S_ERROR	Error	error	Operator tidak valid

g. Transisi DFA untuk Tanda Baca dan Assignment

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	,	S_COMMA	Final	comma	Koma
S0	;	S_SEMICOLON	Final	semicolon	Titik koma
S0	.	S_PERIOD	Final	period	Titik
S0	:	S_COLON	Lanjut/Final	colon	Dapat menjadi : atau :=
S_COLON	=	S_BECOMES	Final	becomes	Assignment :=
S_COLON	selain =	final	Final	colon	Kolon tunggal
S0	(lalu karakter berikutnya bukan *	S_LPAR	Final	lparent	Kurung buka biasa
S0	(lalu karakter berikutnya *	masuk proses komentar	Lanjut	-	Komentar (* ... *)

h. Transisi DFA untuk Identifier dan Keyword

State Saat Ini	Input	State Berikutnya	Status	Token Keluaran	Keterangan
S0	huruf	S_KEY_INPUT	Lanjut	-	Mulai membaca kandidat ident/keyword

S_KEY_INPUT	huruf	S_KEY_INPUT	Lanjut	-	Huruf terus dikumpulkan
S_KEY_INPUT	spasi atau akhir input	S_KEY_CLASSIFY	Lanjut	-	Siap klasifikasi
S_KEY_CLASSIFY	lexeme cocok salah satu keyword	S_KEYWORD	Final	nama keyword	Token dikembalikan sebagai string keyword itu sendiri
S_KEY_CLASSIFY	lexeme tidak cocok keyword	S_IDENT	Final	ident	Identifier biasa
S_KEY_INPUT	selain huruf, spasi, atau akhir input	S_ERROR	Error	error	Pola tidak diterima implementasi saat ini

2.3. Struktur Program Program

Program lexical analyzer ini diimplementasikan menggunakan bahasa pemrograman C++ dan terdiri dari lima file yang memiliki tanggung jawab berbeda. Pembagian file ini dilakukan supaya file terstruktur dan mudah dibaca serta dikembangkan.

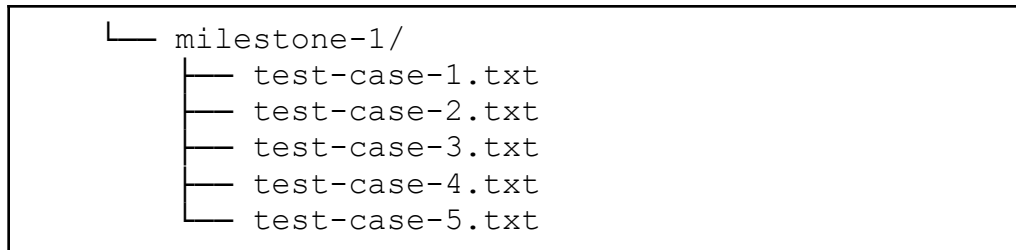
2.3.1. Struktur File Program

Milestone 1 Tugas Besar ini diimplementasikan dengan struktur sebagai berikut

```

GTW-Tubes-IF2224
├── bin
│   └── lexer
├── doc/
│   └── Laporan-1-GTW.pdf
├── src/
│   ├── dfa.h
│   ├── lexer.cpp
│   ├── lexer.h
│   ├── main.cpp
│   └── token.h
└── test

```



2.3.2. Penjelasan Tiap File

Implementasi Lexical Analyzer disimpan dalam folder src melalui beberapa modul dalam lima file berikut.

File	Peran
token.h	Mendefinisikan struktur data Token yang terdiri dari dua field, yaitu type dan value. File ini juga berisi fungsi helper tokenHasValue() yang mengecek apakah suatu token perlu ditampilkan beserta nilainya saat output ditulis.
dfa.h	Mendefinisikan seluruh state DFA sebagai enum.
lexer.h	Mendefinisikan class Lexer beserta seluruh atribut dan deklarasi method-nya.
lexer.cpp	Implementasi logika lexical analyzer sebagai method yang dideklarasikan di lexer.h.
main.cpp	Entry point program yang membaca input dan menulis output.

2.4. Implementasi Lexical Analyzer

Implementasi lexical analyzer terpusat pada file lexer.cpp. Proses analisis dimulai dari fungsi tokenize() yang membaca source code karakter per karakter menggunakan dua fungsi dasar, yaitu peek() dan advance(). Fungsi peek() digunakan untuk melihat karakter di posisi saat ini tanpa menggeser posisi baca, sedangkan advance() membaca karakter dan sekaligus menggeser posisi ke karakter berikutnya.

Pada setiap iterasi, tokenize() memeriksa karakter pertama untuk menentukan fungsi read* mana yang akan dipanggil. Berikut adalah pembagian fungsinya.

- `readNumber()` dipanggil ketika karakter yang dibaca adalah digit (0–9). Fungsi ini membaca seluruh digit yang berurutan dan menghasilkan token `intcon`. Jika setelah digit ditemukan karakter titik (.) diikuti digit lagi, fungsi melanjutkan pembacaan dan menghasilkan token `realcon`. Jika titik tidak diikuti digit, fungsi menghasilkan token `error`.
- `readPunctuation()` dipanggil pada fungsi `tokenize()` ketika karakter yang dibaca adalah salah satu dari tujuh kemungkinan, yaitu `','`, `'.'`, `':'`, `':'`, `'('`, `'['`, atau `']`. Ketika menerima karakter tersebut, fungsi akan mengubah state program menjadi `S_COMMA`, `S_PERIOD`, `S_SEMICOLON`, `S_COLON`, `S_LPARENT`, `S_LBRACK`, atau `S_RBRACK`. Setelah memetakan karakter masukan ke token dalam bahasa Arion yang sesuai, fungsi akan mengembalikan token dengan type dan value yang sesuai.
- `readOperator()` dipanggil oleh fungsi `tokenize()` ketika karakter yang sedang dibaca merupakan bagian dari operator aritmatika atau perbandingan, yaitu `+`, `-`, `*`, `/`, `=`, `<`, atau `>`. Fungsi ini bertugas membaca satu atau lebih karakter secara berurutan untuk membentuk token operator yang valid sesuai dengan transisi DFA yang telah didefinisikan. Jika kombinasi karakter membentuk operator yang valid (misalnya `<` diikuti `>` menjadi `neq`), maka fungsi akan memanggil `advance()` untuk mengonsumsi karakter kedua dan menghasilkan token yang tepat. Jika karakter tunggal sudah cukup untuk membentuk token (misalnya `<` yang tidak diikuti `=` atau `>` menjadi `lss`), fungsi langsung mengembalikan token tersebut. Fungsi ini juga menangani validasi error handling; misalnya jika lexer membaca karakter `=` tunggal tanpa diikuti `=` lagi, fungsi akan memutus pembacaan dan mengembalikan token `error`, karena simbol assignment yang valid pada bahasa Arion adalah `:=` (ditangani oleh `readPunctuation`) dan simbol equal adalah `==`.
- `readComment(char ch)` dipanggil oleh fungsi `tokenize()` ketika karakter (`ch`) yang sedang dibaca adalah `{` dan `*` (setelah karakter sebelumnya `"( "`). Kedua perbedaan karakter yang dibaca di atas menghasilkan dua state berbeda. Ketika `ch` adalah `{`, sebuah variabel value bertipe data string menambahkan semua karakter setelahnya secara iteratif sampai bertemu karakter `}` atau `'\0'`. Jika karakter terakhir `'\0'`, token mengembalikan nilai `error`. Sebaliknya saat karakter membaca

}, token mengembalikan “komentar” dengan nilai isinya. Kasus kedua adalah saat `ch` adalah `*`, sebuah variabel `value` bertipe data string menambahkan semua karakter setelahnya secara iteratif sampai bertemu karakter `*` atau `'\0'`. Jika karakter terakhir `'\0'`, token mengembalikan nilai error. Sebaliknya saat karakter membaca `*`, karakter harus membaca setelah `*` bila) maka token mengembalikan komentar dengan isinya dan bila selain) maka `*` masih di dalam komentar dan melanjutkan ke state awal setelah membaca `*` pertama. Pengimplementasian fungsi ini diharuskan jika membuka komen dengan { maka ditutup dengan } dan jika dibuka (* maka ditutup dengan *).

- `readString()` dipanggil oleh fungsi `tokenize()` ketika karakter yang sedang dibaca adalah `'`. Saat memasuki kondisi tersebut, state `ONE_QUOTE` muncul menandakan pengulangan dimulai sampai dibaca karakter-karakter tertentu. Saat dibaca karakter `'` dalam state tersebut, karakter setelah `'` harus dibaca bila setelahnya adalah `'` maka `value` ditambahkan `'` karena `'` setelahnya sebagai escape karakter. Sebaliknya, jika karakter setelah `'` selain `'` maka state `CHAR` atau `STRING` ditentukan dari ukuran `value`-nya. Bila ukuran `value` adalah 1 maka dikembalikan `CHAR`, ukuran adalah `value` 0 maka dikembalikan `STRING`, dan ukuran `value` lebih dari 1 maka dikembalikan `STRING`.
- `classifyKeyword(string val)` dipanggil oleh `readIdentOrKeyword()` dengan mengirim `string val`. `classifyKeyword()` mengidentifikasi string yang disimpan oleh `readIdentOrKeyword()` dan merubahnya menjadi token. String yang diterima akan dicocokkan dengan dictionary yang dimiliki pada fungsi. Jika diterima, fungsi telah mengidentifikasi keyword. Fungsi akan mengembalikan token sesuai diksi dan merubah lexer menuju state `S_KEYWORD`. Jika tidak diterima oleh dictionary (string tidak dikenali), fungsi mengidentifikasinya sebagai indent. Fungsi akan mengembalikan token berupa string yang diterima dan merubah lexer menuju state `S_IDENT`.
- `readIdentOrKeyword(char ch)` dipanggil ketika karakter yang sedang dibaca adalah karakter alfabet (`[A-Z + a-z]`). Saat menerima karakter tersebut, lexer memasuki state `S_KEY_INPUT` untuk melakukan identifikasi token secara menyeluruh. Identifikasi token dilakukan selama lexer membaca karakter

alfanumerik ([A-Z + a-z + 0-9]). Karakter yang dibaca akan disimpan dalam suatu variabel var. Token yang diterima kemudian dinormalisasi menjadi lower case (case-insensitive). Lexer kemudian memasuki state S_CLASSIFY dan memanggil fungsi classifyKeyword(var).

2.5. Kode Program

readNumber()

```
Token Lexer::readNumber()
{
    string val = "";
    if (currentState == S_MINUS)
    {
        val += '-';
    }

    currentState = S_INT;

    while (isdigit(peek()))
    {
        val += advance();
    }

    if (peek() == '.')
    {
        if (isdigit(source[pos + 1]))
        {
            currentState = S_REAL_DOT;
            val += advance();

            if (!isdigit(peek()))
            {
                currentState = S_ERROR;
                return Token{"error", val};
            }
            while (isdigit(peek()))
```

```

        {
            val += advance();
        }
        currentState = S_REAL;
        return Token({"realcon", val});
    }
    return Token({"intcon", val});
}

if (!isalnum(peek()))
    return Token({"intcon", val});

while (isalnum(peek()))
{
    val += advance();
}
return Token({"unknown", val});
}

```

readComment(ch)

```

Token Lexer::readComment(char ch)
{
    string val = "";
    if (ch == '{')
    {
        advance();
        while (peek() != '}' && peek() != '\0')
        {
            val += peek();

            advance();
        }
        if (peek() == '\0')
        {
            currentState = S_ERROR;
            return Token{"error", "unterminated comment"};
        }
        currentState = S_CMT1;
    }
}

```

```

        advance();
        return Token{"comment", val};
    }
    else
    {
        advance();
        while (peek() != '\0')
        {
            if (peek() == '*')
            {
                currentState = S_CMT2_STAR;
                advance();
                if (peek() == ')')
                {
                    currentState = S_CMT2;
                    advance();
                    return Token{"comment", val};
                }
                val += '*';
                if (peek() != '\0')
                {
                    val += peek();
                    advance();
                }
            }
            else
            {
                val += peek();
                advance();
            }
        }
        currentState = S_ERROR;
        return Token{"error", "unterminated comment"};
    }
};

```

readString()

```
Token Lexer::readString()
{
    string val = "";
    advance();
    while (true)
    {
        char ch = peek();
        if (ch == '\\0' || ch == '\\n')
        {
            return Token{"error", "unterminated string"};
        }

        if (ch == '\\')
        {
            advance();
            if (peek() == '\\')
            {
                advance();
                val += '\\';
            }
            else
            {
                if (val.empty())
                {
                    currentState = S_QUOTE;
                    return Token{"string", ""};
                }
                if (val.size() == 1)
                {
                    currentState = S_CHAR;
                    return Token{"charcon", "'" + val + "'"};
                }
                currentState = S_QUOTE;
                return Token{"string", "'" + val + "'"};
            }
        }
        else
        {
            val += peek();
        }
    }
}
```



```

        advance();
    }
}
};

```

readOperator(ch)

```

Token Lexer::readOperator(char ch)
{
    string val(1, ch);

    switch (ch)
    {
        case '+':
            currentState = S_PLUS;
            return Token{"plus", val};
        case '-':
            if (isdigit(peek()) && (currentState != S_INT &&
currentState != S_REAL))
            {
                currentState = S_MINUS;

                return readNumber();
            }
            currentState = S_MINUS;

            return Token{"minus", val};
        case '*':
            currentState = S_TIMES;
            return Token{"times", val};
        case '/':
            currentState = S_RDIV;
            return Token{"rdiv", val};
        case '=':
            currentState = S_EQL1;
            if (peek() == '=')
            {
                val += advance(); // Consume '=' yang kedua
            }

```

```

        currentState = S_EQL2;
        return Token{"eq1", val};
    }
    currentState = S_ERROR;
    return Token{"error", val};
case '<':
    currentState = S_LESS;
    if (peek() == '>')
    {
        val += advance();
        currentState = S_NOTEQUAL;
        return Token{"neq", val};
    }
    else if (peek() == '=')
    {
        val += advance(); // Consume '='
        currentState = S_LESSEQUAL;
        return Token{"leq", val};
    }
    return Token{"lss", val};
case '>':
    currentState = S_GREATER;
    if (peek() == '=')
    {
        val += advance(); // Consume '='
        currentState = S_GREATEREQUAL;
        return Token{"geq", val};
    }
    return Token{"gtr", val};
default:
    currentState = S_ERROR;
    return Token{"error", val};
}
};

```

readPunctuation()

```

Token Lexer::readPunctuation(char ch)
{

```

```
string val(1, ch);

switch (ch)
{
case ',':
    currentState = S_COMMA;
    return Token{"comma", val};
case ';':
    currentState = S_SEMICOLON;
    return Token{"semicolon", val};
case '.':
    currentState = S_PERIOD;
    return Token{"period", val};
case ':':
    currentState = S_COLON;
    if (peek() == '=')
    {
        val += advance();
        currentState = S_BECOMES;
        return Token{"becomes", val};
    }
    return Token{"colon", val};

case ')':
    currentState = S_RPAR;
    return Token{"rparent", val};
case '[':
    currentState = S_LBRACK;
    return Token{"lbrack", val};
case ']':
    currentState = S_RBRACK;
    return Token{"rbrack", val};
default:
    currentState = S_ERROR;
    return Token{"error", val};
}

};
```

classifyKeyword()

```
string Lexer::classifyKeyword(string val)
{
    string lower = val;
    transform(lower.begin(), lower.end(), lower.begin(),
::tolower);
    std::map<string, string> keywords = {"const", "constsy"},
{"case", "casesy"}, {"var", "varsy"}, {"function", "functionsy"},
{"for", "forsy"}, {"array", "arraysy"}, {"record", "recordsy"},
{"repeat", "repeatsy"}, {"if", "ifsy"}, {"while", "whilesy"},
{"end", "endsy"}, {"else", "elsesy"}, {"of", "ofsy"}, {"do",
"dosy"}, {"downto", "downtosy"}, {"procedure", "proceduresy"},
{"program", "programsy"}, {"until", "untilsy"}, {"begin",
"beginsy"}, {"type", "typesy"}, {"then", "thensy"}, {"to",
"tosy"}, {"not", "notsy"}, {"and", "andsy"}, {"or", "orsy"},
{"div", "idiv"}, {"mod", "imod"};
    // "array", "begin", "case", "const", "do", "downto", "else",
"end", "for", "function", "if", "of", "procedure", "program",
"record", "repeat", "then", "to", "type", "until", "var", "while"

    auto it = keywords.find(lower);
    if (it != keywords.end())
    {
        currentState = S_KEYWORD;
        return it->second;
    }

    currentState = S_IDENT;
    return "ident";
};
```

readIdentOrKeyword()

```
Token Lexer::readIdentOrKeyword(char ch)
{
    string val = "";
    val += ch;
    currentState = S_KEY_INPUT;
```

```

        advance();

        while (isalnum(peek()))
        {
            val += peek();
            advance();
        }

        string lower = val;
        transform(lower.begin(), lower.end(), lower.begin(),
::tolower);
        currentState = S_KEY_CLASSIFY;
        return Token{classifyKeyword(val), lower};
    };

```

tokenize()

```

vector<Token> Lexer::tokenize()
{
    vector<Token> tokens;

    while (peek() != '\0')
    {
        if((currentState != S_INT && currentState != S_REAL) &&
peek() != '-')
        {
            currentState = S0;
        }

        char ch = peek();

        // skip whitespace
        if (isspace(ch))
        {
            advance();
            continue;
        }
    }
}

```

```
// read Number
if (isdigit(ch))
{
    tokens.push_back(readNumber());
}

// read Identifier(variable name) or Keyword(beginsy,
termausuk -> MOD, AND, div, termasuk semua yang pakai string)
else if (isalpha(ch))
{
    tokens.push_back(readIdentOrKeyword(ch));
}
// read String
else if (ch == '\\')
{
    tokens.push_back(readString());
}

// read Comment
else if (ch == '{')
{
    tokens.push_back(readComment(ch));
}
else if (ch == '(')
{
    advance();
    if (peek() == '*')
    {
        tokens.push_back(readComment(ch));
    }
    else
    {
        // advance(); // Cukup maju untuk '('
        currentState = S_LPAR;
        tokens.push_back({"lparent", "("});
    }
}
}
```

```
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
        ||
            ch == '=' || ch == '<' || ch == '>')
        {
            advance();
            tokens.push_back(readOperator(ch));
        }

        else if (ch == ',' || ch == '.' || ch == ';' || ch == ':')
        ||
            ch == ')' || ch == '[' || ch == ']')
        {
            advance();
            tokens.push_back(readPunctuation(ch));
        }

        // Karakter tidak dikenal
        else
        {
            tokens.push_back(Token{"Token", string(1, ch)});
            advance();
        }
    }
    return tokens;
};
```

BAB III

PENGUJIAN

3.1. Metode Pengujian

Serangkaian pengujian terhadap lexer untuk memvalidasi kemampuan program dalam mengenali berbagai jenis token pada bahasa Arion. Pengujian dilakukan dengan memberikan masukan lima berkas masukan .txt yang unik agar dapat memastikan pemenuhan fungsionalitas program secara keseluruhan. Masing-masing kasus uji memuat berbagai bentuk input, seperti identifier, keyword, operator, delimiter, dan kondisi error.

3.2. Hasil Pengujian

Setiap hasil pengujian disajikan dalam tabel di bawah, lengkap dengan input, output, dan analisisnya.

3.2.1. Kasus Uji 1

Tujuan:

Menguji tokenisasi bilangan integer dan real yang valid, serta perilaku lexer pada bilangan campuran alfanumerik (12abc), leading zero (011), dan real negatif berantai (-6.7 - 7.2). Berbeda dari Kasus Uji 6 yang membahas e-notation dan 10.-5, kasus ini berfokus pada interaksi state S_INT/S_REAL terhadap operator minus dan karakter tak valid.

Input:

```
program IntRealTest;
var a, b, c, d: real;
begin
  a := 42;
  b := 3.14;
  c := 12.;
  d := -5;
  b := -6.7 - 7.2;
  c := 12abc;
end.
```

Bukti Input:


```
test > milestone-1 > test-case-1.txt
1  program IntRealTest;
2  var a, b, c, d: real;
3  begin
4      a := 42;
5      b := 3.14;
6      c := 12.;
7      d := -5;
8      b := -6.7 - 7.2;
9      c := 12abc;
10 end.
```

Output:

```
programsy
ident (intrealtest)
semicolon
varsy
ident (a)
comma
ident (b)
comma
ident (c)
comma
ident (d)
colon
ident (real)
semicolon
beginsy
ident (a)
becomes
intcon (42)
semicolon
ident (b)
becomes
realcon (3.14)
semicolon
ident (c)
becomes
intcon (12)
period
semicolon
ident (d)
becomes
intcon (-5)
semicolon
ident (b)
becomes
realcon (-6.7)
```

```
minus
realcon (7.2)
semicolon
ident (c)
becomes
error (12abc)
semicolon
endsy
period
```

Bukti Output:

```
test > milestone-1 > test-case-1-output.txt
1  programsy
2  ident (intrealtest)
3  semicolon
4  varsy
5  ident (a)
6  comma
7  ident (b)
8  comma
9  ident (c)
10 comma
11 ident (d)
12 colon
13 ident (real)
14 semicolon
15 beginsy
16 ident (a)
17 becomes
18 intcon (42)
19 semicolon
20 ident (b)
21 becomes
22 realcon (3.14)
23 semicolon
24 ident (c)
25 becomes
```

Analisis:

42 dan 0 intcon langsung karena !isalnum(peek()) terpenuhi setelah digit habis. 3.14 → realcon karena titik diikuti digit, masuk branch S_REAL_DOT. 12. → titik ditemukan, tapi source[pos+1] adalah ; bukan digit, sehingga branch real tidak diambil dan dikembalikan intcon "12". Titik baru diproses sebagai period pada iterasi tokenize() berikutnya. -5 → saat - dibaca di readOperator, currentState == S0 (bukan S_INT/S_REAL) dan isdigit(peek()) terpenuhi, sehingga langsung masuk readNumber() menghasilkan intcon "-5". -6.7 - 7.2 → token pertama -6.7 berhasil dibaca sebagai realcon dengan mekanisme yang sama. Setelahnya currentState =

S_REAL, sehingga - berikutnya tidak memenuhi kondisi masuk readNumber() dan dikembalikan sebagai minus. 12abc → readNumber() mendeteksi isalnum(peek()) setelah digit selesai, masuk loop akumulasi dan mengembalikan error.

Kesimpulan:

Lexer menangani seluruh variasi int dan real sesuai logika DFA yang diimplementasikan. Trailing dot tidak crash, bilangan negatif dikenali berbasis state, dan input campuran alfanumerik dikembalikan sebagai error untuk ditangani lebih lanjut oleh parser.

3.2.2. Kasus Uji 2

Tujuan:

Kasus uji ini dirancang untuk memvalidasi kemampuan lexer dalam mengenali operator aritmatika serta secara spesifik menguji ketahanan modul pembacaan operator aritmatika tersebut dan perbandingan ganda. Selain itu, skenario ini bertujuan untuk mengevaluasi sistem penanganan error (error handling) ketika lexer dihadapkan pada operator yang tidak valid dalam bahasa Arion, seperti penggunaan simbol sama dengan tunggal (=) maupun kombinasi operator yang berdempetan secara tidak wajar.

Input:

```
program OperatorTest;
var
  x: integer;
  y: real;
  s: string;
  a: integer;
  b: integer;
  c: integer;
  d: integer;
  e: integer;
  z: integer;
  check: boolean;
begin
  z = a + b - c * d / e;
```

```
check := x == y;
check := x <> y;
check := x <= y;
{ == error test operator }
check := a >- b
check := a >< b
check := a = b
end.
```

Bukti Input:

```
program OperatorTest;
var
  x: integer;
  y: real;
  s: string;
  a: integer;
  b: integer;
  c: integer;
  d: integer;
  e: integer;
  z: integer;
  check: boolean;
begin
  z = a + b - c * d / e;
  check := x == y;
  check := x <> y;
  check := x <= y;
  { == error test operator }
  check := a >- b;
  check := a >< b;
  check := a = b;
end.
```

Output yang Diharapkan:

Program diharapkan mampu menghasilkan daftar token, lengkap dengan pencatatan status error tanpa mengalami crash saat menemui karakter tidak valid.

Output:

```
programsy
ident (operator test)
semicolon
varsy
ident (x)
colon
ident (integer)
semicolon
ident (y)
colon
ident (real)
semicolon
ident (s)
colon
ident (string)
semicolon
ident (a)
colon
ident (integer)
semicolon
ident (b)
colon
ident (integer)
semicolon
ident (c)
colon
ident (integer)
semicolon
ident (d)
colon
ident (integer)
semicolon
ident (e)
colon
ident (integer)
semicolon
ident (z)
colon
ident (integer)
semicolon
ident (check)
colon
ident (boolean)
semicolon
```

beginsy
ident (z)
error (=)
ident (a)
plus
ident (b)
minus
ident (c)
times
ident (d)
rdiv
ident (e)
semicolon
ident (check)
becomes
ident (x)
eq
ident (y)
semicolon
ident (check)
becomes
ident (x)
neq
ident (y)
semicolon
ident (check)
becomes
ident (x)
leq
ident (y)
semicolon
comment (== error test operator)
ident (check)
becomes
ident (a)
gtr
minus
ident (b)
ident (check)
becomes
ident (a)
gtr
lss
ident (b)
ident (check)
becomes

ident (a)
error (=)
ident (b)
endsy
period

Bukti Output:

```
programsy      You, 4 hours ago • tes  
ident (operatortest)  
semicolon  
varsy  
ident (x)  
colon  
ident (integer)  
semicolon  
ident (y)  
colon  
ident (real)  
semicolon  
ident (s)  
colon  
ident (string)  
semicolon  
ident (a)  
colon  
ident (integer)  
semicolon  
ident (b)  
colon  
ident (integer)  
semicolon  
ident (c)  
colon  
ident (integer)  
semicolon
```

Analisis:

1. Lexer sukses membaca seluruh keyword (seperti programsy, varsy, beginsy, endsy), nama variabel (identifier), dan tipe data dasar tanpa mengalami anomali karakter ganda di akhir kata. Ini

membuktikan perbaikan pembacaan alphanumeric sudah berjalan dengan sempurna.

2. Pada pengujian baris $z = a + b$, program mendeteksi tanda $=$ sebagai error karena dalam bahasa Arion, assignment menggunakan $:=$ dan operator komparasi menggunakan $==$. Yang terpenting, setelah mengeluarkan token error, program tetap melanjutkan kompilasi untuk membaca sisa ekspresi aritmatikanya.
3. Pada kombinasi anomali $>-$ dan $><$, lexer tidak mengalami kebingungan. Sesuai aturan automata, karakter tersebut dievaluasi satu per satu dan dipecah menjadi token gtr ($>$) diikuti oleh minus ($-$), serta gtr ($>$) diikuti lss ($<$). Selain itu, program masih tetap menghasilkan output, meskipun pada 3 baris terakhir program tidak diakhiri dengan *semicolon* ($;$).

Kesimpulan:

Secara keseluruhan, *lexical analyzer* telah mampu menjalankan proses tokenisasi secara komprehensif. Modul pembacaan operator yang diimplementasikan terbukti bekerja dengan baik dalam menguraikan simbol-simbol operasional dan sukses menahan berbagai anomali karakter tanpa mengganggu siklus kompilasi secara paksa.

3.2.3. Kasus Uji 3

Tujuan:

Untuk menguji kemampuan lexer dalam mengenali karakter-karakter yang disesuaikan dalam edge case pada string, char, dan komentar.

Input:

```
(*Tes case untuk uji ketahanan program lexer*)  
  
Program robustness;  
  
var  
  
{uji string dan char}
```



```
q := '';  
s := '''';  
t := 'it''s';  
v := 'string\n biasa';  
w := 'belum ditutup  
{input x := ''}  
x := ''';
```

end.

Bukti Input:

```
test > milestone-1 > test-case-3.txt  
1  (*Tes case untuk uji ketahanan program lexer*)  
2  
3  Program robustness;  
4  
5  var  
6  
7  {uji string dan char}  
8      q := '';  
9      s := '''';  
10     t := 'it''s';  
11     v := 'string\n biasa';  
12     w := 'belum ditutup  
13     {input x := ''}  
14     x := ''';  
15  
16  (* comment edge case *)  
17  { komentar tidak ditutup  
18  (* komentar bintang tidak ditutup  
19  
20  (* comment dengan karakter spesial di dalamnya *)  
21  { komentar dengan } di tengah lanjutan }  
22  (* komentar dengan *) di tengah lanjutan *)  
23  
24  end.
```

Output yang Diharapkan:

Program dapat memberikan output yang sesuai pada uji-uji kasus ketika string kosong, char ‘, escape ‘, string tidak ditutup dengan error, dan input “” hasilnya error.

Output:

```
comment (Tes case untuk uji ketahanan program lexer)  
programsy  
ident (robustness)  
semicolon  
varsy
```

comment (uji string dan char)
ident (q)
becomes
string (")
semicolon
ident (s)
becomes
charcon ("")
semicolon
ident (t)
becomes
string ('it's')
semicolon
ident (v)
becomes
string ('string\n biasa')
semicolon
ident (w)
becomes
error (unterminated string)
comment (input x := "")
ident (x)
becomes
error (unterminated string)
endsy
period

Bukti Output:

```

❏ output-3.txt
1  comment (Tes case untuk uji ketahanan program lexer)
2  programsy
3  ident (robustness)
4  semicolon
5  varsy
6  comment (uji string dan char)
7  ident (q)
8  becomes
9  string ('')
10 semicolon
11 ident (s)
12 becomes
13 charcon ('')
14 semicolon
15 ident (t)
16 becomes
17 string ('it's')
18 semicolon
19 ident (v)
20 becomes
21 string ('string\n biasa')
22 semicolon
23 ident (w)
24 becomes
25 error (unterminated string)
26 comment (input x := '')
27 ident (x)
28 becomes
29 error (unterminated string)
30 endsy
31 period

```

Analisis:

Program memberikan output “” pada kasus string kosong. Lalu memberikan output ‘ ‘ ‘ sesuai dengan input char ‘. Selanjutnya pada kasus uji “It’s”, program memberikan output juga sesuai dengan ‘ di dalam string. Setelah itu, string yang tidak ditutup membuat program memberikan output error. Terakhir pada uji kasus “”, program memberikan output error.

Kesimpulan:

Program dapat memberikan output sesuai output harapan maka program dapat menangani kasus-kasus tersebut dengan benar. Program dapat menangani edge cases pada string, char, dan komentar.

3.2.4. Kasus Uji 4

Tujuan:

Menguji pembacaan punctuation pada kondisi normal dan edge case, terutama perbedaan : dan :=, penanganan = tunggal yang invalid, nested punctuation.

Input:

```

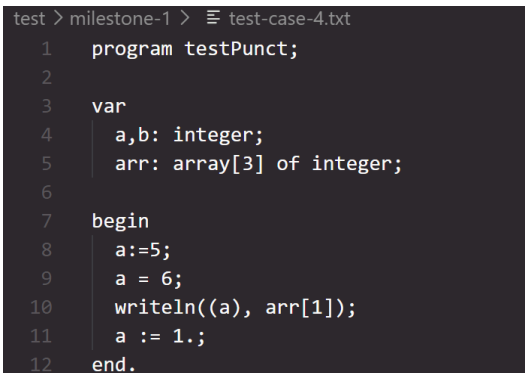
program testPunct;

var
  a,b: integer;
  arr: array[3] of integer;

begin
  A :=5;
  a = 6;
  writeln((a), arr[1]);
  a := 1.;
end.

```

Bukti Input:



```

test > milestone-1 > test-case-4.txt
1  program testPunct;
2
3  var
4    a,b: integer;
5    arr: array[3] of integer;
6
7  begin
8    a:=5;
9    a = 6;
10   writeln((a), arr[1]);
11   a := 1.;
12  end.

```

Output yang Diharapkan:

Lexer diharapkan mengenali seluruh punctuation valid dengan benar. Pada `a ::= 5;`, token yang diharapkan adalah `ident(a)`, `error`, `colon`, `becomes`, `intcon(5)`, dan `semicolon`. Pada `a = 6;`, `=` tunggal juga harus menjadi `error`. Baris `writeln((a), arr[1]);` harus terbaca normal, sedangkan `a := 1.;` masih valid jika menghasilkan `intcon(1)`, `period`, dan `semicolon`.

Output:

```

programsy
ident (testpunct)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)

```

```
semicolon  
ident (arr)  
colon  
arraysy  
lbrack  
intcon (3)  
rbrack  
ofsy  
ident (integer)  
semicolon  
beginsy  
ident (a)  
becomes  
intcon (5)  
semicolon  
ident (a)  
error  
intcon (6)  
semicolon  
ident (writeln)  
lparent  
lparent  
ident (a)  
rparent  
comma  
ident (arr)  
lbrack  
intcon (1)  
rbrack  
rparent  
semicolon  
ident (a)  
becomes  
intcon (1)  
period  
semicolon  
endsy  
period
```

Bukti Output:

```
test > milestone-1 > test-case-4-output.txt
```

```
1  programsy
2  ident (testpunct)
3  semicolon
4  varsy
5  ident (a)
6  comma
7  ident (b)
8  colon
9  ident (integer)
10 semicolon
11 ident (arr)
12 colon
13 arraysy
14 lbrack
15 intcon (3)
16 rbrack
17 ofsy
18 ident (integer)
19 semicolon
20 beginsy
21 ident (a)
22 error
23 colon
24 becomes
25 intcon (5)
```

Analisis

Output sebenarnya sudah sesuai harapan. Deklarasi variabel dan array berhasil ditokenisasi dengan benar, = tunggal konsisten ditandai sebagai error, dan rangkaian ::= berhasil dipecah tanpa mengganggu token berikutnya. Selain itu, nested punctuation pada `writeln((a), arr[1]);` dan kasus `a := 1.;` juga telah ditangani dengan benar.

Kesimpulan:

Lexer sudah cukup baik dalam menangani punctuation normal, punctuation berurutan, dan simbol invalid.

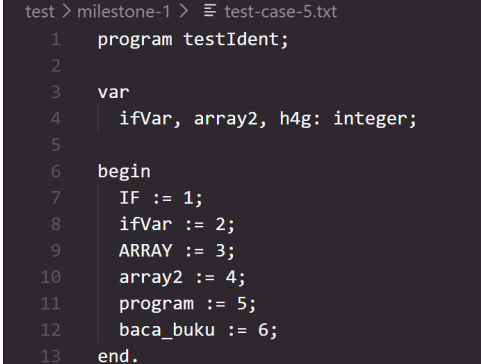
3.2.5. Kasus Uji 5

Tujuan:

Untuk menguji kemampuan lexer dalam mendeteksi kesalahan leksikal saat menemukan input yang tidak valid dan terus melakukan pengecekan hingga akhir.

Input:

```
program testIdent;  
  
var  
    ifVar, array2, h4g: integer;  
  
begin  
    IF := 1;  
    ifVar := 2;  
    ARRAY := 3;  
    array2 := 4;  
    program := 5;  
    baca_buku := 6;  
end.
```

Bukti Input:

```
test > milestone-1 > test-case-5.txt  
1  program testIdent;  
2  
3  var  
4  | ifVar, array2, h4g: integer;  
5  
6  begin  
7  | IF := 1;  
8  | ifVar := 2;  
9  | ARRAY := 3;  
10 | array2 := 4;  
11 | program := 5;  
12 | baca_buku := 6;  
13 end.
```

Output yang Diharapkan:

Lexer seharusnya membaca IF sebagai ifsy, ARRAY sebagai arraysy, program sebagai programsy, sedangkan ifVar, array2, dan h4g tetap sebagai ident. Untuk baca_buku, lexer tidak boleh menganggapnya sebagai satu ident utuh karena _ tidak valid, sehingga harus muncul error atau unknown pada bagian tersebut.

Output:

```
programsy  
ident (testident)  
semicolon  
varsy  
ident (ifvar)  
comma  
ident (array2)
```

```
comma
ident (h4g)
colon
ident (integer)
semicolon
beginsy
ifsy
becomes
intcon (1)
semicolon
ident (ifvar)
becomes
intcon (2)
semicolon
arraysy
becomes
intcon (3)
semicolon
ident (array2)
becomes
intcon (4)
semicolon
programsy
becomes
intcon (5)
semicolon
ident (baca)
error
ident (buku)
becomes
intcon (6)
semicolon
endsy
period
```

Bukti Output:


```
test > milestone-1 > test-case-5-output.txt
```

```
1  programsy
2  ident (testident)
3  semicolon
4  varsy
5  ident (ifvar)
6  comma
7  ident (array2)
8  comma
9  ident (h4g)
10 colon
11 ident (integer)
12 semicolon
13 beginsy
14 ifsy
15 becomes
16 intcon (1)
17 semicolon
18 ident (ifvar)
19 becomes
20 intcon (2)
21 semicolon
22 arraysy
23 becomes
24 intcon (3)
25 semicolon
```

Analisis:

KKasus uji ini mengecek apakah lexer bisa membedakan keyword dan identifier yang mirip. IF dan ARRAY menguji keyword case-insensitive, ifVar dan array2 menguji identifier yang mirip keyword tapi tetap valid, sedangkan baca_buku menguji kegagalan karena mengandung underscore.

Kesimpulan:

Kasus uji berhasil jika lexer mengklasifikasikan keyword dan identifier dengan benar, tetap mengenali program sebagai keyword, dan menolak baca_buku sebagai identifier valid.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan pada Milestone 1, dapat ditarik beberapa kesimpulan sebagai berikut:

- Program Lexical Analyzer (lexer) untuk bahasa pemrograman Arion telah berhasil diimplementasikan menggunakan bahasa pemrograman C++. Implementasi ini dibangun secara mandiri tanpa menggunakan library atau tools pembantu otomatis, sesuai dengan spesifikasi yang diminta.
- Proses pengenalan token berhasil dimodelkan menggunakan prinsip Deterministic Finite Automata (DFA). Meskipun terdapat sedikit penyesuaian pada kode program (pembacaan keyword menggunakan string utuh alih-alih per karakter) demi menjaga kode agar tidak terlalu kompleks dan mudah dimodifikasi, alur kerja utama tetap merepresentasikan perilaku DFA dengan akurat.
- Lexer mampu membaca *source code* mentah dari kiri ke kanan dan mengklasifikasikannya ke dalam 52 jenis token yang didefinisikan untuk bahasa Arion secara komprehensif.
- Berdasarkan skenario pengujian yang telah dilakukan, program terbukti kuat dan stabil. Program mampu menangani berbagai *edge cases* seperti *string* atau komentar yang tidak ditutup, angka negatif, maupun *error* pada operator tanpa menyebabkan *crash* pada program secara keseluruhan.

4.2. Saran

Untuk pengembangan selanjutnya, terutama untuk milestone berikutnya, terdapat beberapa saran yang dapat diterapkan:

- Saat ini, program sudah mampu mendeteksi error (seperti unterminated string atau operator tidak valid) tanpa berhenti beroperasi secara paksa. Ke depannya, informasi *error* dapat dikembangkan dengan menambahkan posisi baris dan kolom terjadinya kesalahan pada *source code* agar *error handling* menjadi jauh lebih informatif bagi pengguna.
- truktur program saat ini yang sudah terpisah berdasarkan tanggung jawabnya

(*token.h*, *dfa.h*, *lexer.cpp*, dll) harus terus dijaga dan ditingkatkan pemisahannya.

Modularitas ini sangat disarankan untuk mempermudah integrasi dengan berbagai tahapan kompilasi selanjutnya.

- Seiring dengan bertambahnya kompleksitas program pada *milestone* selanjutnya, struktur data dan mekanisme *buffer* dalam pembacaan karakter berulang (*advance* dan *peek*) dapat dioptimalkan lebih lanjut untuk mempercepat proses kompilasi pada program Arion yang berskala besar.

DAFTAR PUSTAKA

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson.
- Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Gate Smashers. (2020). Lec-3: Lexical Analysis in Compiler Design with Examples. YouTube. Tersedia di: <https://www.youtube.com/watch?v=BgNdtk9h8Ok>
- University of Illinois. (2009). Lexical Analysis. CS421 Summer 2009 Lectures. Diakses dari <https://courses.grainger.illinois.edu/cs421/su2009/lectures/05-lexical-analysis.pdf>

LAMPIRAN

Tautan Repository GitHub

Seluruh kode sumber program dapat diakses melalui repositori berikut.

<https://github.com/andraalanna/GTW-Tubes-IF2224-2026>